

```
....1st base case. Returning ['()()']
...2nd base case. Returning ['()()']
..2nd base case. Returning ['()()']
.2nd base case. Returning ['(() )', '()()']
2nd base case. Returning ['(() )', '()()']
Results: ['(() )', '()()']
```

The `getBalancedParens()` function ❶ requires one argument, the number of pairs of parentheses, when called by the user. However, it needs to pass additional information in the arguments to its recursive calls. These include the number of opening parentheses that remain to be added (`openRem`), the number of closing parentheses that remain to be added (`closeRem`), and the current balanced parentheses string being built (`current`). Both `openRem` and `closeRem` start as the same value as the `pairs` argument, and `current` starts as the blank string. An `indent` argument is used only for the debugging output to show the program's level of recursive function call.

The function first checks the number of opening and closing parentheses remaining to be added ❷. If both are 0, we've reached the first base case, and the string in `current` is finished. Since the `getBalancedParens()` function returns a list of strings, we put `current` in a list and return it ❸.

Otherwise, the function continues on to the recursive case. If possible opening parentheses remain ❹, the function calls `getBalancedParens()` with an opening parenthesis added to the current argument. If more closing parentheses are remaining than opening parentheses ❺, the function calls `getBalancedParens()` with a closing parenthesis added to the current argument. This check ensures that an unmatched closing parenthesis won't be added, as this would make the string unbalanced, such as the second closing parenthesis in `() )`.

After these recursive cases is an unconditional base case that returns all the strings returned from the two recursive function calls (and, of course, the recursive function calls made by these recursive function calls, and so on) ❻.